Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Janamaejayan V S	Register No.	3122235001057

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

1. Aim Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA)
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions

2. Dataset Description

English Handwritten Characters Dataset
Download Link: Dataset

- Total samples: 3,410
- Number of classes: 62 (0–9, A–Z, a–z)
- Image type: Grayscale

3. Preprocessing Steps

- The English Handwritten Characters dataset was loaded into a Pandas DataFrame for preprocessing.
- Grayscale images were resized to a fixed dimension, normalized to the range [0,1], and flattened into feature vectors.
- Input features (X) and target labels (y) were separated, and categorical labels were encoded into numerical form.
- A stratified train–test split was performed to preserve class distribution across all 62 classes.
- Feature scaling was applied using the StandardScaler technique prior to training the MLP model.

4. Implementation Details

- Implemented the Single-Layer Perceptron (PLA) from scratch using a step activation function and the perceptron weight update rule.
- Extended the PLA to multi-class classification using the One-vs-Rest (OvR) strategy.
- Implemented a Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions trained using backpropagation.
- Performed hyperparameter tuning on the MLP by experimenting with learning rate, optimizer, batch size, and hidden layer configurations.

5. Visualizations

5.1 Single Layer Perceptron (SLP)

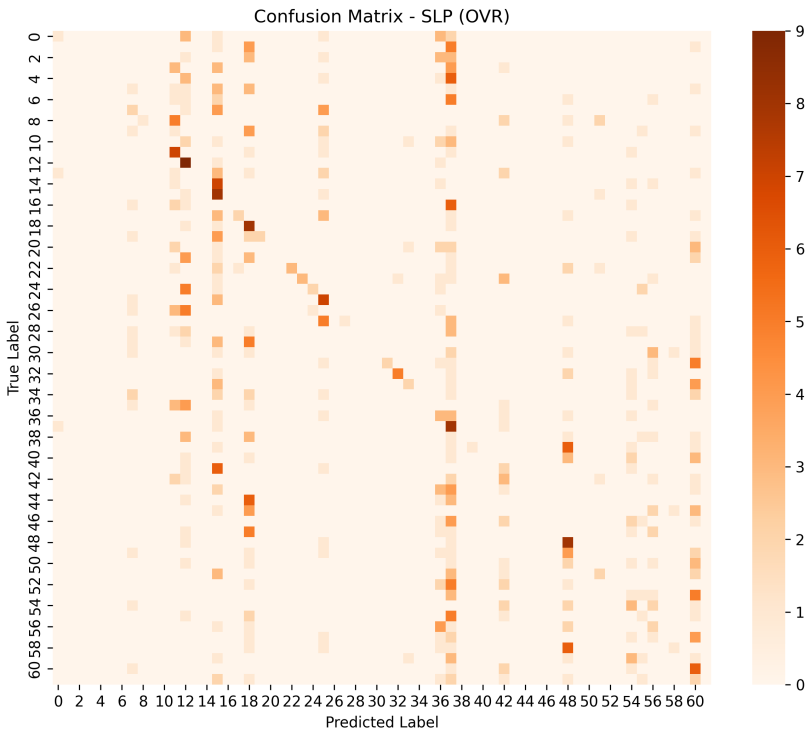

Figure 1: Confusion Matrix (SLP)

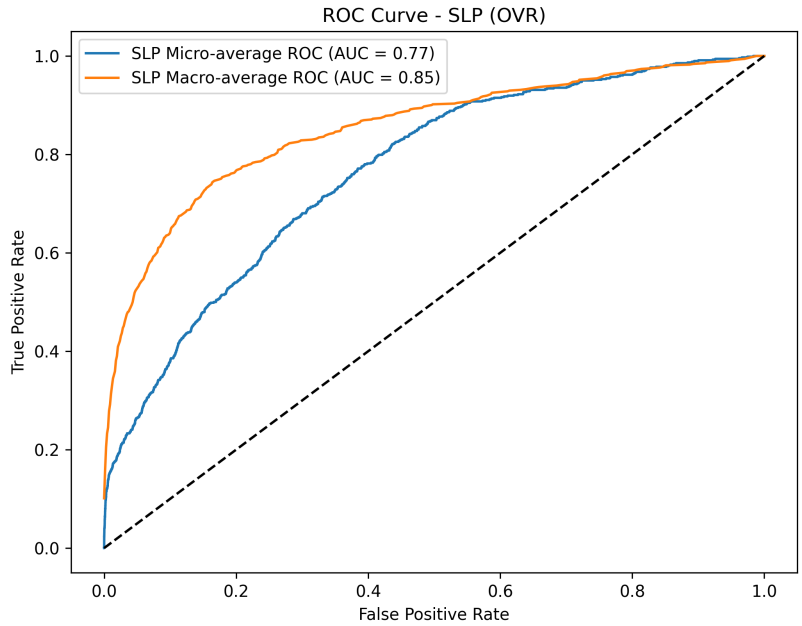

Figure 2: ROC (SLP)

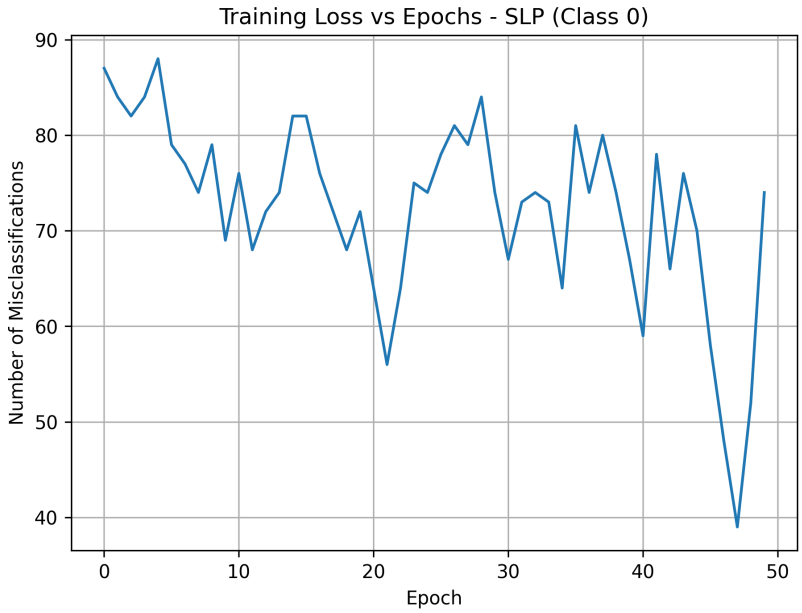

Figure 3: Training vs Epoch(SLP)

5.2 Multi Layer Perceptron (MLP)

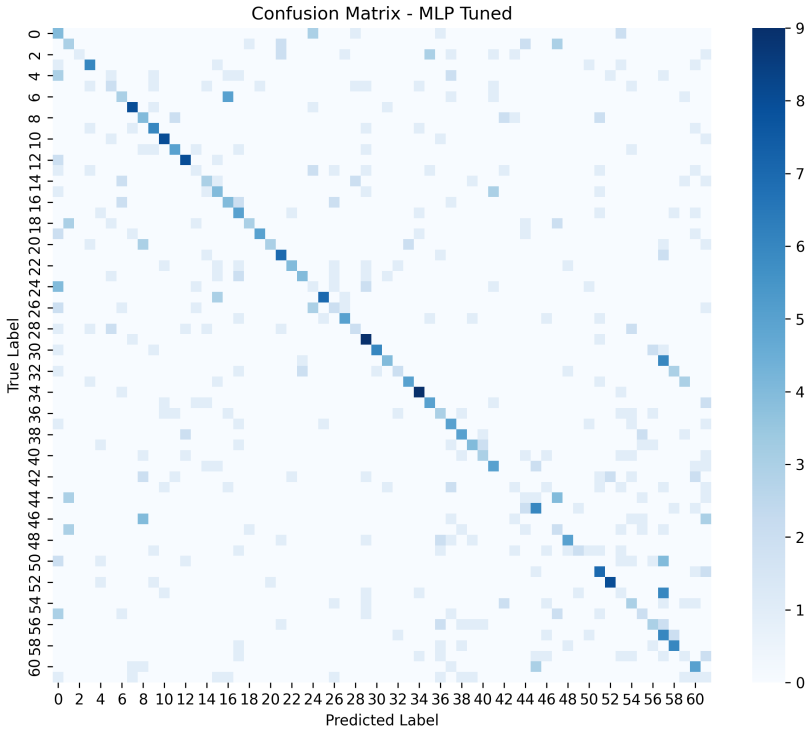

Figure 4: Confusion Matrix (MLP)

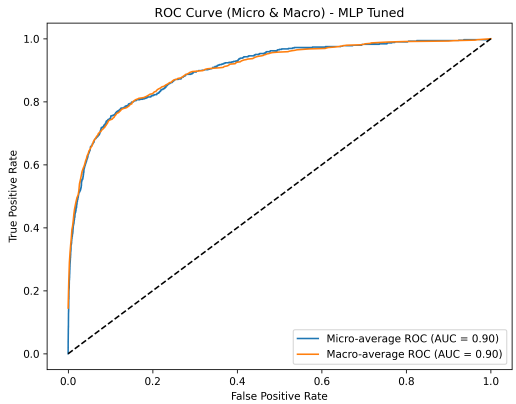

Figure 5: ROC (MLP)

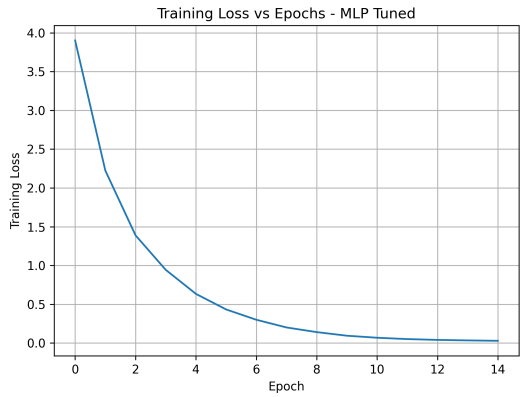

Figure 6: Training vs Epoch (MLP)

6. Performance Tables

Table 1: Overall Performance Comparison between PLA and MLP

Model	Accuracy (%)	Precision	Recall	F1-score
PLA (OvR)	15.10	0.2112	0.1481	0.1202
MLP (Tuned)	38.12	0.4071	0.3812	0.3801

Table 2: Hyperparameter Tuning Results for MLP

Hidden Layers	Activation	Optimizer	Learning Rate	Batch Size	Accuracy (%)
1 (128)	ReLU	SGD	0.01	32	38.12
2 (256–128)	ReLU	Adam	0.001	64	35.63
3 (512–256–128)	Tanh	Adam	0.0005	64	31.67

Table 3: Training Convergence Comparison

Model	Epochs	Final Training Loss	Convergence Behavior
PLA	20	84	Non-convergent
MLP (Tuned)	15	0.02834	Stable convergence

7. A/B Experiment Comparison

Final tuned hyperparameters for MLP

- Hidden Layers: 1(128)
- Activation: Relu
- Optimizer: SGD
- Learning Rate: 0.01
- Batch Size: 32

Strengths and weakness of PLA vs MLP:

The Single-Layer Perceptron (PLA) is simple, fast to train, and computationally efficient, making it suitable for basic binary classification problems; however, it can only learn linear decision boundaries and struggles with complex, nonlinearly separable data such as handwritten character recognition. In contrast, the Multilayer Perceptron (MLP) can model nonlinear relationships using hidden layers and backpropagation, resulting in better performance on complex multiclass tasks,

but it requires careful hyperparameter tuning, more computational resources, and longer training time.

Effect of hyperparameter tuning on convergence and accuracy:

Hyperparameter tuning significantly improves both convergence and accuracy of the MLP model. Selecting an appropriate learning rate ensures stable weight updates and prevents divergence, while choosing the right optimizer (such as Adam) accelerates convergence. Adjusting the number of hidden layers and neurons enhances the model's ability to learn complex nonlinear patterns, leading to better classification performance. Proper batch size selection also improves training stability and generalization. Overall, tuning helps the model converge faster and achieve higher accuracy compared to default settings.

8. Observation Questions

Why does PLA underperform compared to MLP?

PLA underperforms compared to MLP because it can only learn linear decision boundaries and cannot capture complex nonlinear patterns in data. Handwritten character recognition requires modeling intricate feature relationships, which PLA cannot handle. MLP, with hidden layers and nonlinear activation functions, can learn these complex patterns and therefore achieves better accuracy.

Which hyperparameter had the most impact on MLP performance?

The learning rate and optimizer had the most impact on MLP performance rather than increasing hidden layers. Adding more hidden layers did not significantly improve accuracy and sometimes led to unstable convergence or overfitting. Proper tuning of learning rate and optimizer (such as Adam) contributed more to stable convergence and better performance.

How does optimizer choice affect convergence?

The choice of optimizer affects how quickly and smoothly the model converges. Optimizers like Adam adapt the learning rate and usually provide faster and more stable convergence, while SGD may converge more slowly and be sensitive to the learning rate. In our experiment, SGD led to better convergence compared to ADAM.

Does increasing hidden layers always improve performance?

No, increasing hidden layers does not always improve performance. While deeper networks can model more complex patterns, they may also lead to overfitting, increased training time, and unstable convergence if the dataset is small or not sufficiently complex. In our experiment, adding more hidden layers did not necessarily improve accuracy and sometimes reduced performance.

How can overfitting be detected and mitigated?

Overfitting can be detected when training accuracy is high but test accuracy is much lower, indicating poor generalization. It can be mitigated by reducing model complexity, using regularization or dropout, applying early stopping, and properly tuning hyperparameters.